Loop alignment: Self-organized Weight Transpose in Predictive Coding through Independent Hebbian Plasticity.

Anonymous Author(s)
Affiliation
Address
email

Abstract

Backpropagation (BP) requires feedback (FB) weights to equal the transpose of feedforward (FF) weights at every layer and at every step. This issue is called the weight transport problem, and has been considered to be biologically implausible. Neural circuits are not built from anatomically paired FF/FB axons but from asymmetric projections that form recurrent loops shared across many computations. In multi-nuclei circuits (e.g., thalamo-cortical, cortico-cerebellar, basal-ganglia loops), the return path traverses entirely distinct nuclear structures, making any mechanism for maintaining synchronized weight transposes anatomically absent. Predictive coding (PC), originally proposed as a biologically plausible substitute for BP, silently inherits this constraint despite its biological motivation. Therefore, we propose two PC variants that eliminate it entirely. **PC-DH** (Dual Hebbian plasticity model) updates FF and FB weights by independent local Hebbian rules with no inter-pathway coordination. **PC-RFB** (Random FeedBack) fixes the FB weight with a random matrix, testing the robustness of the learning mechanism. Both match baseline PC accuracy ($\approx 85\%$ on MNIST). We identify a phenomenon we termed **Loop Alignment**: the spontaneous convergence of FF and FB weights toward a transpose relationship, driven by the structure of the inference loop. We then prove that it holds exactly at every training step in both models. To test whether Loop Alignment holds in the multi-nuclei setting, we conduct a circuit-sharing experiment: two networks learn different tasks (Net 1: MNIST; Net 2: Fashion-MNIST) over a shared pathway, where each network's FB is routed through the other's FF weights. Even when the entire shared pathway is plastic and driven simultaneously by both tasks, each network solves its task independently and FF/FB alignment is stronger than in the isolated case. Loop Alignment explains how the brain can learn effectively without weight transport, and support multi-task learning over shared asymmetric circuits.

1 Introduction

Neural circuits are organized as recurrent loops, not as anatomically paired feedforward (FF) and feedback (FB) channels. In cortico-cortical projections, FF and FB axons originate from different cell types, occupy distinct laminar compartments, and exhibit plasticity rules that differ in form, magnitude, and time scale [Thomson and Bannister, 2003, Bastos et al., 2012, Gerstner et al., 2014]. The asymmetry runs deeper in multi-nuclei circuits, for example, in thalamo-cortical, cortico-cerebellar, and basal-ganglia loops, the return path traverses entirely distinct nuclear structures populated by separate cell populations with no shared plasticity signal. In both cases, what is shared is the anatomical channel—a bundle of asymmetric projections connecting two populations—not a co-

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

ordinated, functionally paired set of axons. The same pathway participates in perception, attention, motor prediction, memory retrieval, and context modulation, often simultaneously and in different roles [Rigotti et al., 2013]. There is no strict anatomical pairing of individual FF and FB axons into coordinated monosynaptic loops.

This biological reality places the *weight transport problem* at the center of any biologically plausible learning theory. Backpropagation (BP) [Rumelhart et al., 1986] requires the FB weight matrix at every layer to be the exact transpose of the corresponding FF matrix at every training step. In a brain where bidirectional pathways traverse multiple relay nuclei, engage separate cell populations, and serve overlapping functional roles, no mechanism exists to enforce—or even plausibly approximate—this synchronized transpose relationship. The weight transport constraint is not merely unverified; it is structurally incompatible with the known anatomy of both cortical and subcortical circuits.

Predictive coding (PC) has emerged as the leading biologically plausible alternative to BP [Rao and Ballard, 1999, Friston, 2005, 2010, Whittington and Bogacz, 2017]. In PC, each cortical layer generates top-down predictions; discrepancies are encoded by local error neurons; and both representation and error activities are iteratively refined through bidirectional message passing before any weight update occurs. Whittington and Bogacz [2017] showed that PC weight updates at inference equilibrium approximate BP gradients, making PC a principled route to gradient-based learning without global error broadcast. Yet PC inherits the weight transport problem: its standard formulation requires $W_i^{\mathrm{FB}} = (W_i^{\mathrm{FF}})^\top$ at every layer and every training step—precisely the constraint that cortical and subcortical anatomy makes implausible.

We therefore asked whether the weight transport problem can be dissolved by PC's own inference loop dynamics, without any explicit coordination between FF and FB pathways. The key insight, which emerges from building our models, is that PC's iterative inference loop is itself an implicit weight-alignment engine: weight symmetry is an attractor that inference dynamics naturally reach, not a constraint to be imposed from outside.

Approach. Our primary model, **PC-DH** (Dual Hebbian Plasticity), updates FF and FB weights by fully independent local Hebbian rules. No information is shared between the two update steps. We prove that the FF and FB updates are exact transposes of one another at every step, so the two weight matrices converge to a transpose relationship as the contribution of the random initialization is washed out.

To isolate the inference loop's contribution from any FB learning rule, we introduce **PC-RFB**: the FB pathway is a fixed random matrix, never updated. This removes even the possibility of coordinated adaptation between pathways, capturing the extreme case in which FF and FB axons belong to entirely separate anatomical systems. Even under this maximally constrained condition, FF weights spontaneously align with the random FB structure (cosine similarity ≈ 0.9). We derive the explicit inference-driven drift term responsible for this alignment, showing that the inference loop alone without any FB plasticity is sufficient to drive FF weights toward an approximate transpose relationship.

Circuit multiplexing as functional validation. If FB pathways need not carry precisely calibrated, transpose-matched signals, they can be shared, repurposed, and overlapping—as multi-nuclei anatomy directly implies. We test this by constructing a circuit-sharing scenario that instantiates this topology: two PC networks (Net 1 learning MNIST; Net 2 learning Fashion-MNIST) share a single bidirectional pathway, with each network's FB signal routed through the other network's continuously learned FF weights. This produces an extreme entanglement: each network's FB pathway is reshaped at every step by the other network's learning on a different task. Loop Alignment persists under this condition—both networks reach $\approx 85\%$ accuracy on their respective tasks while maintaining strong FF–FB cosine similarity. In the fully plastic variant, where every weight in the shared loop is updated simultaneously by both networks' Hebbian rules, alignment quality is *higher* than in the single-task isolated case, suggesting that global plasticity actively supports rather than threatens Loop Alignment.

Contributions.

1. **PC-DH and an exact Loop Alignment result.** We propose PC-DH model and prove that two independently updated local Hebbian rules produce cumulative updates that are exact transposes of each other at every step (Section 4.1).

2. **PC-RFB and the inference-driven drift mechanism.** We introduce PC-RFB and derive the explicit drift term, generated by the inference loop, that drives FF weights toward the random FB scaffold without any FB plasticity (Section 3.3).

3. **Multi-task circuit sharing on a fully plastic shared loop.** We demonstrate that two networks sharing bidirectional pathways, and solving distinct tasks maintain Loop Alignment, with alignment quality strengthened when all shared weights are simultaneously learnable (Section 5.3).

2 Related work

Predictive coding. Rao and Ballard [1999] introduced hierarchical PC as a model of visual cortex. The free-energy principle [Friston, 2005, 2010, Friston and Kiebel, 2009] generalized PC to a unified theory of perception and learning. Bastos et al. [2012] provided neurophysiological evidence mapping superficial cortical layers to bottom-up error signals and deep layers to top-down predictions. Whittington and Bogacz [2017] proved that PC's local Hebbian updates, at convergence, recover BP gradients. Millidge et al. [2022] review the field. All prior formulations assume $W_i^{\mathrm{FB}} = (W_i^{\mathrm{FF}})^\top$; weight transport has been acknowledged but not resolved within PC. A complementary challenge in scaling PC networks is depth degradation: performance drops sharply beyond five to seven layers due to exponentially imbalanced prediction errors across layers [Qi et al., 2025]. While the present work focuses on the weight transport problem in shallow-to-moderate architectures, combining loop alignment with precision-weighted updates [Qi et al., 2025] is a natural direction for extending biologically plausible learning to deep architectures.

Feedback alignment. Lillicrap et al. [2016] demonstrated that in BP, fixed random FB weights suffice for learning because FF weights align with the random FB matrix over training (Feedback Alignment, FA). Extensions include direct FA [Nøkland, 2016] and sign-symmetry studies [Liao et al., 2016]. These operate within BP's single-pass computational graph. In PC-DH, FF and FB weights are learned by indiopendent Hebbian rules but self-organize toward a transpose relationship through PC's bidirectional inference dynamics. PC-RFB is inspired by the FA, but we find that the alignment mechanism is qualitatively different from original FA.

Equilibrium propagation, target propagation, and forward-only learning. Equilibrium propagation [Scellier and Bengio, 2017] computes gradients through energy perturbations. Recent work [Laborieux and Zenke, 2024] addressed its analogous weight-symmetry requirement via Jacobian homeostasis. Contrastive Hebbian learning [Hinton, 2002], target propagation [Lee et al., 2015], dendritic microcircuits [Guerguiev et al., 2017, Sacramento et al., 2018], and forward-forward [Hinton, 2022] employ local objectives but require distinct computational phases or auxiliary structures, while our models operate within the standard single-phase PC inference loop.

3 Models

We use the supervised PC model of Whittington and Bogacz [2017] as our baseline. The network has three layers with sizes 784–100–10 for representation neurons and two layers of error neurons (sizes 100 and 10). We omit nonlinear activations, adaptive optimizers, and normalization from this study, ensuring that any observed alignment is attributable to the core PC dynamics and Hebbian plasticity, not to auxiliary deep-learning machinery.

3.1 Baseline predictive coding

At each layer i:

$$\boldsymbol{\varepsilon}_i = \mathbf{x}_{i+1} - \mathbf{x}_i W_i^{\mathrm{FF}}, \tag{1}$$

$$\Delta \mathbf{x}_i = \eta(-\boldsymbol{\varepsilon}_{i-1} + \boldsymbol{\varepsilon}_i W_i^{\mathrm{FB}}), \tag{2}$$

$$\Delta W_i^{\mathrm{FF}} = \alpha(\mathbf{x}_i^\top \boldsymbol{\varepsilon}_i), \tag{3}$$

$$W_i^{\mathrm{FB}} = (W_i^{\mathrm{FF}})^\top. \tag{4}$$

For each input, 50 inference loops update $\mathbf{x}_i$ with weights fixed; weights are then updated via Eq. (3).

3.2 PC-DH: Predictive Coding with Dual Hebbian Plasticity

PC-DH is our primary model. FF and FB weights are independent matrices, each governed by its own local Hebbian rule with no information shared between them. After inference with Eq. (1) and Eq. (2), both weight matrices update independently as follows:

$$\Delta W_i^{\mathrm{FF}} = \alpha(\mathbf{x}_i^\top \boldsymbol{\varepsilon}_i), \tag{5}$$

$$\Delta W_i^{\mathrm{FB}} = \alpha(\boldsymbol{\varepsilon}_i^\top \mathbf{x}_i). \tag{6}$$

Eq. (6) contains no reference to W_i^{FF}. No transpose operation, weight copying, or synchronized update is required at any point. Both updates use the same neural activities $\mathbf{x}_i$ and $\boldsymbol{\varepsilon}_i$ produced by the same inference loop.

3.3 PC-RFB: Predictive Coding with fixed Random FB

In PC-RFB, W_i^{FB} is replaced by a fixed random matrix B_i, drawn Xavier-normal at initialization and never updated:

$$\Delta \mathbf{x}_i = \eta(-\boldsymbol{\varepsilon}_{i-1} + \boldsymbol{\varepsilon}_i B_i). \tag{7}$$

FF weights are updated as in Eq. (5). The random matrix B_i models the scenario in which the FB pathway traverses relay nuclei with no-plasticity or no task-relevant structure: if the effective FB connectivity is a product of random matrices across nuclei, the result is itself a random matrix. Crucially, FF and FB do not share weight, so there is no monosynaptic reciprocal connection. PC-RFB is meant to isolate the contribution of the inference loop by removing FB learning.

4 Theoretical analysis

4.1 Loop Alignment is structurally guaranteed in PC-DH

The key observation is that the FF and FB Hebbian rules in PC-DH use the same neural activities at every step. Accumulating updates across T training steps,

$$W^{\mathrm{FF}}(T) = W^{\mathrm{FF}}(0) + \alpha \sum_{t=1}^{T} \mathbf{x}_i(t)^\top \boldsymbol{\varepsilon}_i(t), \tag{8}$$

$$W^{\mathrm{FB}}(T) = W^{\mathrm{FB}}(0) + \alpha \sum_{t=1}^{T} \boldsymbol{\varepsilon}_i(t)^\top \mathbf{x}_i(t). \tag{9}$$

At every single step, the FF and FB outer products are exact transposes of one another:

$$\boldsymbol{\varepsilon}_i(t)^\top \mathbf{x}_i(t) \;=\; \big(\mathbf{x}_i(t)^\top \boldsymbol{\varepsilon}_i(t)\big)^\top. \tag{10}$$

This identity holds pointwise, so it distributes over the entire sum:

$$W^{\mathrm{FF}}(T) - W^{\mathrm{FF}}(0) \;=\; \big(W^{\mathrm{FB}}(T) - W^{\mathrm{FB}}(0)\big)^\top. \tag{11}$$

The accumulated updates are therefore always in exact transpose. The only obstacle to $W^{\mathrm{FF}}(T) \approx (W^{\mathrm{FB}}(T))^\top$ is the initial weights $W^{\mathrm{FF}}(0)$ and $W^{\mathrm{FB}}(0)$. As training progresses, these random initializations are washed out by the growing Hebbian updates.

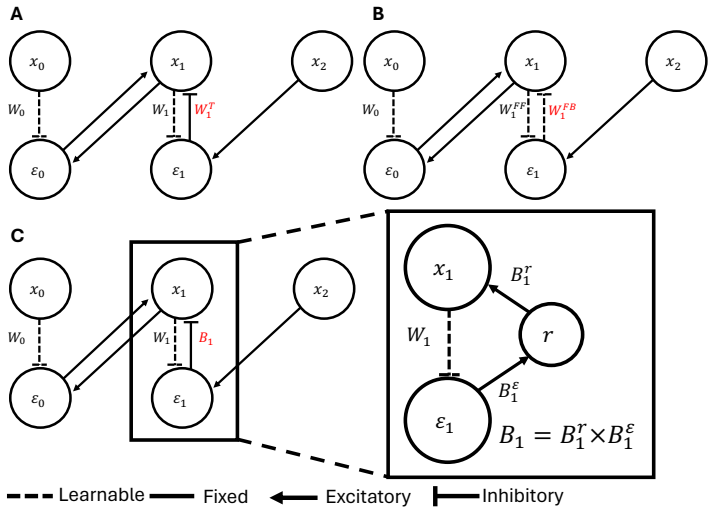

Figure 1: **Model architectures.** Circles: neural populations ($\mathbf{x}$: representation neurons; ε: error neurons). Dashed arrows: learnable weights; solid arrows: fixed weights. **(A)** Baseline PC: FB weights are the transpose of FF weights, encoding the weight transport problem. **(B)** PC-DH: FF and FB are independent matrices updated by fully independent Hebbian rules; no coordination. **(C)** PC-RFB: FB is a fixed random matrix B, modeling FB pathways through asymmetric relay nuclei (right inset: an effective B as the product of random matrices through a relay).

Inference update rule. Under predictive-coding inference dynamics, the representation $\mathbf{x}_i$ is adjusted at each inference step to simultaneously reduce the local prediction error ε_{i-1} (the error for which layer i is responsible) and incorporate the top-down feedback signal $\varepsilon_i B_i$ arriving via the fixed random scaffold. A single discrete inference step therefore reads

$$\tilde{\mathbf{x}}_i \;=\; \mathbf{x}_i \;-\; \eta\,\varepsilon_{i-1} \;+\; \eta\,\varepsilon_i\,B_i. \tag{12}$$

The first correction pulls $\mathbf{x}_i$ toward the FF prediction from below; the second incorporates the feedback-weighted error from above. Together they define the inference trajectory whose fixed point is the posterior estimate of $\mathbf{x}_i$ given both neighbouring layers.

Deriving the three-term decomposition. After the inference step, the FF Hebbian update is applied using the updated representation $\tilde{\mathbf{x}}_i$ as the pre-synaptic term:

$$\Delta W^{\mathrm{FF}} \;=\; \alpha\,(\tilde{\mathbf{x}}_i^{\top}\,\varepsilon_i). \tag{13}$$

Substituting (12) into (13) and expanding linearly:

$$\begin{aligned}\Delta W^{\mathrm{FF}} \;&=\; \alpha\,\big(\mathbf{x}_i - \eta\,\varepsilon_{i-1} + \eta\,\varepsilon_i\,B_i\big)^{\top}\varepsilon_i \\ &=\; \alpha\,\mathbf{x}_i^{\top}\,\varepsilon_i \;-\; \alpha\,\eta\,\varepsilon_{i-1}^{\top}\,\varepsilon_i \;+\; \alpha\,\eta\,\big(\varepsilon_i B_i\big)^{\top}\varepsilon_i.\end{aligned} \tag{14}$$

The third term can be simplified as follows:

$$\big(\varepsilon_i\,B_i\big)^{\top}\varepsilon_i \;=\; B_i^{\top}\;(\varepsilon_i^{\top}\,\varepsilon_i) \tag{15}$$

Substituting back into (14) and we obtain:

$$\Delta W^{\mathrm{FF}} \;=\; \underbrace{\alpha\,\mathbf{x}_i^{\top}\varepsilon_i}_{\text{Hebbian}} \;-\; \underbrace{\alpha\,\eta\,\varepsilon_{i-1}^{\top}\varepsilon_i}_{\text{cross-layer decorrelation}} \;+\; \underbrace{\alpha\,\eta\,B_i^{\top}\big(\varepsilon_i^{\top}\,\varepsilon_i\big)}_{\text{alignment drift}}. \tag{16}$$

In the PC-DH model, the same decomposition holds with $B_i = W^{\mathrm{FB}}$, so the alignment drift term becomes $\alpha\eta\,W^{\mathrm{FB}\top}(\varepsilon_i^{\top}\varepsilon_i)$ where W^{FF} is nudged toward $W^{\mathrm{FB}\top}$ by inference, while the symmetric Hebbian rule simultaneously nudges W^{FB} toward $W^{\mathrm{FF}\top}$, driving Loop Alignment from both directions.

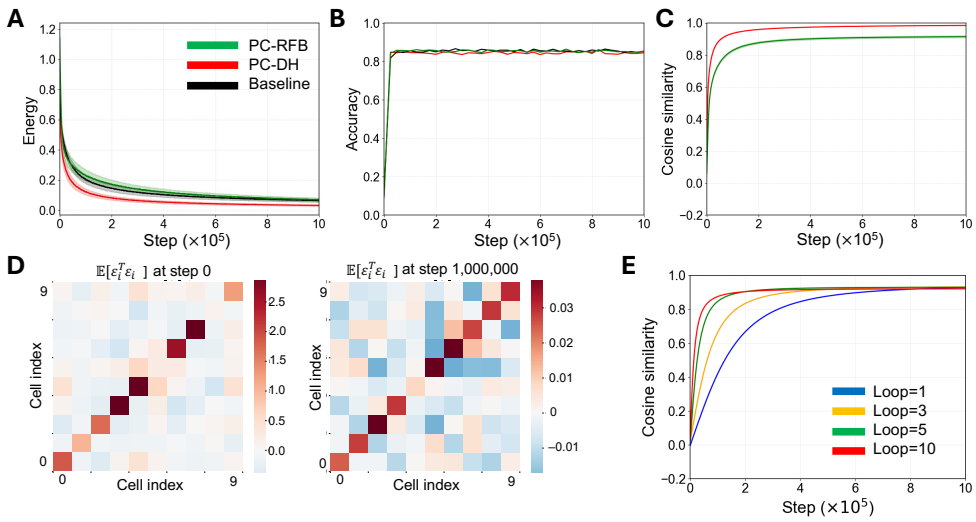

Figure 2: **Single-task learning performance and weight alignment.** Green: PC-RFB; red: PC-DH; black: baseline. Mean over 50 sessions (10^6 steps); shading ± 1 SD. **(A)** Energy. Both proposed models minimize energy as stably as baseline. **(B)** MNIST test accuracy. PC-DH and PC-RFB reach $\approx 85\%$, matching baseline (McNemar, $p > 0.05$). **(C)** FF–FB cosine similarity. PC-DH $\to 1.0$; PC-RFB $\to 0.9$. **(D)** Batch-averaged error auto-correlation $\mathbb{E}[\boldsymbol{\varepsilon}_i^\top \boldsymbol{\varepsilon}_i]$ at start (left) and at convergence (right) of the training in PC-RFB. **(E)** Effect of inference loop count ($L \in \{1, 3, 5, 10\}$) on FF–FB cosine similarity in PC-RFB.

5 Experiments

Setup. All single-network experiments use a 3-layer architecture (784–100–10), Xavier initialization, mini-batches of 20, and 50 inference loops per input. Main experiments: 10^6 steps, 50 sessions; circuit-sharing and robustness experiments: $3{\times}10^6$ steps, 10 sessions. No nonlinear activations, adaptive optimizers, or normalization. The plots show the mean for all sessions (solid line) ± 1 SD (shaded). Full hyperparameters are provided in Appendix A.

5.1 Single-task learning performance

Both PC-DH and PC-RFB achieve monotonic energy minimization and reach $\approx 85\%$ classification accuracy, matching the transpose baseline (Figure 2A–B). Neither model differs significantly from baseline by McNemar's test (Appendix C).

5.2 Weight alignment dynamics

PC-DH. Cosine similarity between FF and FB weights rises from ≈ 0 (random initialization) to ≈ 1.0 over training (Figure 2C). Two independently updated Hebbian matrices converge to a transpose relationship with no explicit coordination.

PC-RFB. Cosine similarity rises from ≈ 0 to ≈ 0.9 (Figure 2C), showing strong partial alignment.

Single-sample outer products exhibit pronounced off-diagonal structure (rank-1 by construction). However, batch-averaged matrices are diagonally dominant at both stages (Figure 2D), confirming that the whiteness approximation underlying Eq. (25) holds throughout learning. We then examine how loop number can affect those alignment. Figure 2E shows FF–FB cosine similarity for PC-RFB under $L \in \{1, 3, 5, 10\}$ inference loops per step. Alignment occurs under all conditions and increases monotonically with L, consistent with the prediction that each inference loop contributes one application of the alignment drift term.

5.3 Multi-task circuit sharing

Circuit structure. To test our proposed models under biologically realistic circuit entanglement, we construct a multi-task circuit-sharing setup (Fig. 3A). Two PC networks, Net 1 (MNIST) and Net 2 (Fashion-MNIST), each maintain their own FF weight matrices $W_i^{\mathrm{Net}\,1}$ and $W_i^{\mathrm{Net}\,2}$, updated by independent local Hebbian rules. The two networks share a single bidirectional pathway at the second layer: the representation/error neurons $\mathbf{x}_i^{\mathrm{Net}\,1}, \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1}, \mathbf{x}_i^{\mathrm{Net}\,2}, \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2}$ form a closed loop in which each network's FB signal is routed through the *other* network's FF weights, sandwiched by two relay matrices $\mathbf{U}$ and $\mathbf{V}$ (Xavier-normal, shared across networks). Biologically, $\mathbf{U}$ can be thought of as the projection from a network's local error population into a shared relay nucleus, while $\mathbf{V}$ returns the processed signal back to the originating network's representation neurons—an abstraction of the input/output projections of any multi-nuclei relay (e.g. thalamus in a thalamo-cortical loop). Because the loop is closed, the two FB composites traverse the loop in opposite directions:

$$W_i^{\mathrm{FB,\,Net}\,1} \;=\; \mathbf{U}\, W_i^{\mathrm{Net}\,2}\, \mathbf{V}, \qquad W_i^{\mathrm{FB,\,Net}\,2} \;=\; \mathbf{V}\, W_i^{\mathrm{Net}\,1}\, \mathbf{U}. \tag{17}$$

Alternating training with passive relay. The two networks are trained in strict *alternation*: each training step is dedicated to a single network. We do not use mini-batching; one image is processed per step. At step t, if the active network is Net 1 we draw $s^{\mathrm{Net}\,1}(t) \sim \mathcal{D}_{\mathrm{MNIST}}$ (image and one-hot label) and clamp Net 1's bottom and top representations; only Net 1 receives an external input. We then run an inference phase of $L_{\mathrm{inf}} = 50$ loops, in which Net 1's $\mathbf{x}^{\mathrm{Net}\,1}, \boldsymbol{\varepsilon}^{\mathrm{Net}\,1}$ are updated by

$$\Delta \mathbf{x}_i^{\mathrm{Net}\,1} = \eta\big(-\boldsymbol{\varepsilon}_{i-1}^{\mathrm{Net}\,1} + \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1}\, \mathbf{U}\, W_i^{\mathrm{Net}\,2}\, \mathbf{V}\big)\,. \tag{18}$$

Net 2's shared-layer activities are not driven by any input during this step; instead, they are computed *passively as relay values* on Net 1's signal path,

$$\mathbf{x}_i^{\mathrm{Net}\,2} = \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1}\, \mathbf{U}, \qquad \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2} = \mathbf{x}_i^{\mathrm{Net}\,2}\, W_i^{\mathrm{Net}\,2} = \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1}\, \mathbf{U}\, W_i^{\mathrm{Net}\,2}, \tag{19}$$

i.e. as the post-$\mathbf{U}$ and post-$W^{\mathrm{Net}\,2}$ stages of the loop traversal $\boldsymbol{\varepsilon}^{\mathrm{Net}\,1} \;\to\; \boldsymbol{\varepsilon}^{\mathrm{Net}\,1}\mathbf{U} \;\to\; \boldsymbol{\varepsilon}^{\mathrm{Net}\,1}\mathbf{U}\, W^{\mathrm{Net}\,2} \;\to\; \boldsymbol{\varepsilon}^{\mathrm{Net}\,1}\mathbf{U}\, W^{\mathrm{Net}\,2}\mathbf{V}$. After the 50 loops complete, the converged values of $\mathbf{x}^{\mathrm{Net}\,1}, \boldsymbol{\varepsilon}^{\mathrm{Net}\,1}$ together with the relay values $\mathbf{x}^{\mathrm{Net}\,2}, \boldsymbol{\varepsilon}^{\mathrm{Net}\,2}$ are used to update the learnable weights of this step (only Net 1's FF in the partial-plasticity case; Net 1's FF together with $\mathbf{U}$ and $\mathbf{V}$ in the fully plastic case; full update equations in Appendix E). The next step is symmetric with Net 2 active and Net 1 serving as the relay; Net 1 and Net 2 use independently shuffled data pools (different random seeds). The procedure is repeated for $3{\times}10^6$ steps.

Partial plasticity (Fig. 3B–D). $\mathbf{U}$ and $\mathbf{V}$ are fixed throughout training. The only learnable weights are the task-specific FF matrices, and only the FF of the active network is updated at each step (Eq. (5)). Despite each network's effective FB being continuously reshaped by the partner network's task-driven learning, both networks successfully minimize energy and learn their tasks (Fig. 3B–C), with Net 1 (MNIST) and Net 2 (Fashion-MNIST) each reaching $\approx 85\%$ accuracy. Loop Alignment is preserved: the cosine similarity between $W_i^{\mathrm{Net}\,1}$ and its effective feedback weight $\mathbf{U}\, W_i^{\mathrm{Net}\,2}\, \mathbf{V}$ converges to ≈ 0.9 (Fig. 3D).

Full plasticity (Fig. 3E–G). On the active network's step, we additionally apply local Hebbian updates at the input ($\mathbf{U}$) and output ($\mathbf{V}$) synapses of the relay, using only the active network's variables and the relay values defined by Eq. (19) (full update equations in Appendix E). The partner network's FF weights are not updated on this step. Both networks learn their tasks without difficulty (Fig. 3E,F). Interestingly, the cosine similarity converges to an even higher value than in the partial-plasticity setting (Fig. 3G), suggesting stronger Loop Alignment in fully plastic circuit sharing.

5.4 Robustness test with random noise injection

To test the robustness of Loop Alignment, we added uniform noise to every element of the FB matrix B_i at each training step, with amplitude $\delta \in \{1\%, 2\%, 3\%, 5\%, 10\%, 20\%, 30\%\}$ of the Xavier scale. With noise injection, the weight quickly drift from the initial weight (Fig. 4A). For $\delta \le 5\%$, accuracy remains $\approx 85\%$ and cosine similarity converges to 0.2–0.4 (Fig. 4B–D); at $\delta = 10\%$ accuracy decreases modestly; at $\delta \ge 20\%$, alignment is largely prevented and accuracy degrades substantially. Notably, learning succeeds at cosine similarities as low as 0.2, showing that exact alignment was not required for effective PC for the MNIST task.

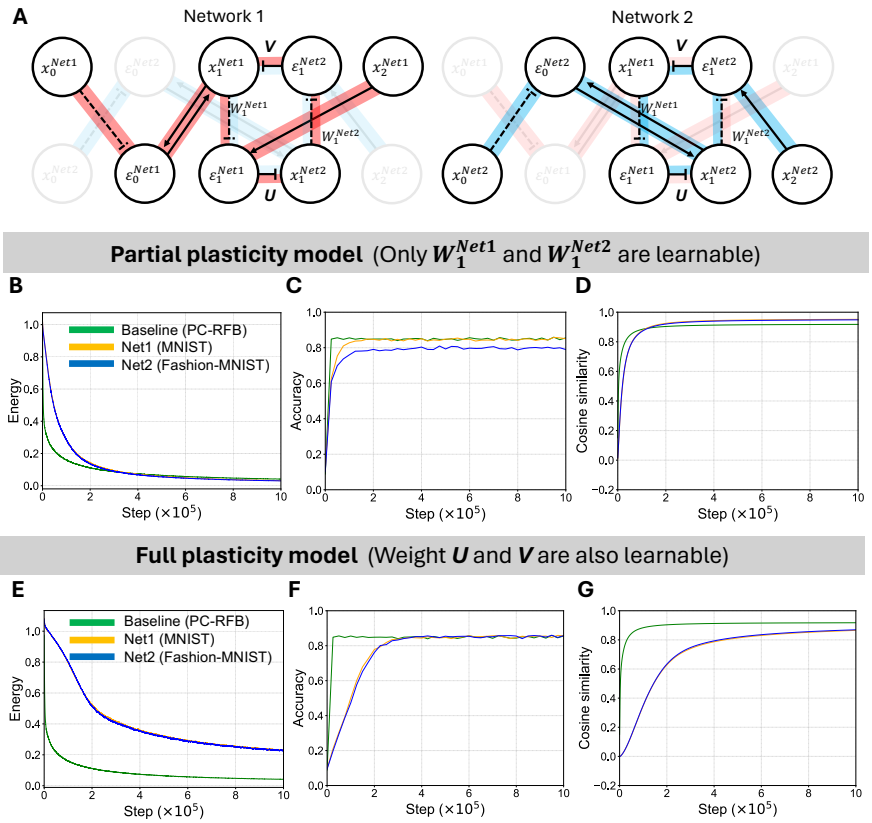

Figure 3: **Multi-task circuit sharing.** Net 1 (MNIST) and Net 2 (Fashion-MNIST) share a bi-directional pathway. Yellow/blue: Net 1/Net 2 in circuit sharing task; green: Baseline with PC-RFB model without circuit share. Mean over 10 sessions (3×10^6 steps); shading ±1 SD. **(A)** Overview of circuit structure. **(B)** Energy of the circuit through learning. **(C)** Test accuracy on each task. Both networks reach $\approx85\%$ despite sharing a continuously changing FB pathway. **(D)** FF–FB cosine similarity. Loop Alignment converges in both networks. **(E–G)** Fully plastic variant: energy, accuracy, and FF–FB cosine similarity when every shared-pathway weight is simultaneously updated by both tasks.

6 Discussion

The inference loop as a weight-alignment engine. The central result of this paper is that PC's inference loop is a self-organizing weight-symmetry mechanism. In PC-DH, this is a structural guarantee because FF and FB Hebbian updates use the same neural activities at the same step, their cumulative sums are exactly transposes of one another (Eq. 10). In PC-RFB, the inference loop iteratively adapts neural activity to the FB weight before each weight update, generating the drift term (Eq. 16; full derivation in Eq. 25) that orients FF weights toward the FB transpose.

Circuit multiplexing and brain theory. Circuit-sharing provides insight into how the brain supports multiple overlapping computations across shared anatomical pathways without destructive interference. Loop Alignment does not require dedicated, segregated circuits, rather, independent Hebbian plasticity in both FF and FB directions is sufficient to maintain weight coherence even when the shared pathway is continuously reshaped by a different task. A key contributor could be the cross-layer decorrelation term $(-\alpha\,\eta\,\boldsymbol{\varepsilon}_{i-1}^{\top}\boldsymbol{\varepsilon}_i)$ in the weight update (Eq. 16): by subtracting correlations between adjacent error layers it suppresses interference, preventing each network's FF weights from drifting toward the other task's error directions even as both tasks drive the same shared weights.

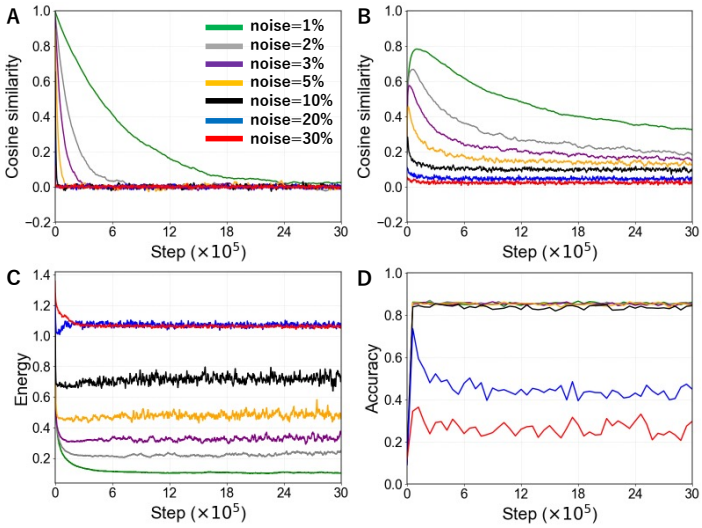

Figure 4: **Robustness test with dynamic FB noise.** PC-RFB model was used to train MNIST with noise injection during the inference loop. **(A)** Cosine similarity between the initial and final FF weights at each noise level, measuring how far the FF weights drifted from initial weight over training. **(B)** FF–FB cosine similarity under continuous additive FB noise ($\delta = 1\%$–30% of Xavier scale). Alignment is stable for $\delta \leq 5\%$ and largely prevented at $\delta \geq 20\%$. **(C)** Learning energy across training steps at each noise level. Energy minimization proceeds normally for low noise and degrades progressively at higher intensities. **(D)** MNIST test accuracy under each noise level. Accuracy is maintained at $\approx 85\%$ for $\delta \leq 5\%$; modest decrease at $\delta = 10\%$; substantial degradation at $\delta \geq 20\%$.

Random FB does not claim that cortex is random. The frozen random W^{FB} in PC-RFB is a worst-case scenario, not a model of cortical anatomy. Cortico-cortical projections have layer-specific termination patterns and cell-type-specific roles [Thomson and Bannister, 2003, Bastos et al., 2012]. The point of PC-RFB is that even at this worst case, learning still works and alignment still emerges; in real cortex, where FB pathways carry useful top-down signals, alignment should be at least as effective. Similarly, our use of fully connected matrices is a first-order approximation, therefore verifying Loop Alignment in sparse, anatomically structured connectivity is an important next step.

Limitations and future directions. We acknowledge that our networks reach $\approx 85\%$ on MNIST, well below modern benchmarks. However, by excluding optimizers, ReLU, batch normalization, and other deep-learning machinery, we ensured that the alignment phenomenon and its robustness cannot be attributed to those auxiliary mechanisms. Scaling Loop Alignment to deeper networks, harder benchmarks, and biologically structured connectivity (Dale's law, sparsity, spiking neurons) is the natural follow-up. Also, our experiments use a deliberately minimal 3-layer network as a clean proof of principle. Whether Loop Alignment persists when FF and FB pathways are updated on different timescales—as cortical anatomy suggests—is an open question, as is a complete theoretical treatment of the alignment improvement in fully plastic circuit sharing. Furthermore, our analysis and experiments focus on networks of 2–4 layers; recent work shows that PC training in deeper architectures requires additional mechanisms to balance error propagation [Qi et al., 2025], and whether loop alignment remains effective in that regime is an open question.

7 Conclusion

Loop Alignment shows that the weight transport problem, long treated as a fundamental obstacle to biologically plausible learning, is dissolved by the dynamics that make predictive coding distinctive. The inference loop is not only a mechanism for minimizing prediction error but also an implicit mechanism to achieve weight symmetry. This mechanism survives under shared and continuously changing circuits, and tolerates the synaptic instability present in real neural circuits.

References

A. M. Bastos, W. M. Usrey, R. A. Adams, G. R. Mangun, P. Fries, and K. J. Friston. Canonical microcircuits for predictive coding. *Neuron*, 76(4):695–711, 2012.

K. Friston. A theory of cortical responses. *Philosophical Transactions of the Royal Society B*, 360 (1456):815–836, 2005.

K. Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2): 127–138, 2010.

K. Friston and S. Kiebel. Predictive coding under the free-energy principle. *Philosophical Transactions of the Royal Society B*, 364(1521):1211–1221, 2009.

W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski. *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, 2014.

J. Guerguiev, T. P. Lillicrap, and B. A. Richards. Towards deep learning with segregated dendrites. *eLife*, 6:e22901, 2017.

G. Hinton. The forward-forward algorithm: Some preliminary investigations. In *arXiv preprint arXiv:2212.13345*, 2022.

G. E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8):1771–1800, 2002.

A. Laborieux and F. Zenke. Improving equilibrium propagation without weight symmetry through Jacobian homeostasis. In *International Conference on Learning Representations*, 2024.

D.-H. Lee, S. Zhang, A. Fischer, and Y. Bengio. Difference target propagation. In *Machine Learning and Knowledge Discovery in Databases*, pages 498–515, 2015.

Q. Liao, J. Leibo, and T. Poggio. How important is weight symmetry in backpropagation? In *AAAI Conference on Artificial Intelligence*, 2016.

T. P. Lillicrap, D. Cownden, D. B. Tweed, and C. J. Akerman. Random synaptic feedback weights support error backpropagation for deep learning. *Nature Communications*, 7:13276, 2016.

B. Millidge, A. Seth, and C. L. Buckley. Predictive coding: a theoretical and experimental review. *arXiv preprint arXiv:2107.12979*, 2022.

A. Nøkland. Direct feedback alignment provides learning in deep neural networks. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

C. Qi, M. Forasassi, T. Lukasiewicz, and T. Salvatori. Towards the training of deeper predictive coding neural networks. *arXiv preprint arXiv:2506.23800*, 2025.

R. P. Rao and D. H. Ballard. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1):79–87, 1999.

M. Rigotti, O. Barak, M. R. Warden, X.-J. Wang, N. D. Daw, E. K. Miller, and S. Fusi. The importance of mixed selectivity in complex cognitive tasks. *Nature*, 497(7451):585–590, 2013.

D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. In *Nature*, volume 323, pages 533–536, 1986.

J. Sacramento, R. P. Costa, Y. Bengio, and W. Senn. Dendritic cortical microcircuits approximate the backpropagation algorithm. *Advances in Neural Information Processing Systems*, 31, 2018.

B. Scellier and Y. Bengio. Equilibrium propagation: Bridging the gap between energy-based models and backpropagation. *Frontiers in Computational Neuroscience*, 11:24, 2017.

A. M. Thomson and A. P. Bannister. Interlaminar connections in the neocortex. *Cerebral Cortex*, 13(1):5–14, 2003.

J. C. Whittington and R. Bogacz. An approximation of the error backpropagation algorithm in a predictive coding network with local Hebbian synaptic plasticity. *Neural Computation*, 29(5): 1229–1262, 2017.

A Hyperparameters

Table 1: Hyperparameters shared across all models and experiments.

Parameter	Value	Notes
Network architecture	784–100–10	Representation neurons
Error neuron sizes	100–10	
Inference loops (main)	50	Varied to 1/3/5/10 in Sec. 5.2
Inference learning rate η	5×10^{-2}	
FF weight learning rate α	1×10^{-2}	
FB weight learning rate (PC-DH)	1×10^{-2}	Equal to FF rate
Mini-batch size	20	Single-task only; 1 for circuit sharing
Weight initialization	Xavier normal	Mean 0
Training steps (main)	10^6	50 sessions
Training steps (circuit/robustness)	3×10^6	10 sessions
MNIST/Fashion-MNIST normalization	$[0, 1]$	Pixel values
Teacher signal	One-hot, clamped	Top-layer representation

B Cosine similarity computation

The FF weight W_i^{FF} has shape $[n, m]$ and the FB weight W_i^{FB} has shape $[m, n]$. To compare element-wise, we transpose W_i^{FF} to shape $[m, n]$ and flatten both matrices to vectors of length $m \times n$. Cosine similarity is computed as $\cos(W_i^{\mathrm{FF}\top}, W_i^{\mathrm{FB}}) = \langle W_i^{\mathrm{FF}\top}, W_i^{\mathrm{FB}} \rangle / (\|W_i^{\mathrm{FF}}\| \, \|W_i^{\mathrm{FB}}\|)$. This equals 1 when $W^{\mathrm{FF}} = W_i^{\mathrm{FB}\top}$ up to a positive scalar factor, 0 when orthogonal, and -1 when anti-aligned.

C Statistical tests

McNemar's test was used to compare classification performance between model pairs on the same test set. For each test sample, we recorded whether each model predicted correctly, constructed a 2×2 contingency table from discordant pairs, and computed the McNemar χ^2 statistic. Significance level: $\alpha = 0.05$.

D Alignment drift: complete derivation

This section gives the complete derivation of the alignment drift term, complementing the sketch in Section 4.

Setting and notation. At layer i, let $\mathbf{x}_i \in \mathbb{R}^n$ and $\boldsymbol{\varepsilon}_i \in \mathbb{R}^m$ be the representation and error vectors (row vectors), $W_i^{\mathrm{FF}} \in \mathbb{R}^{n \times m}$ the FF weight, and $B_i \in \mathbb{R}^{m \times n}$ the fixed random FB matrix. The prediction error and inference update read

$$\boldsymbol{\varepsilon}_i = \mathbf{x}_{i+1} - \mathbf{x}_i \, W_i^{\mathrm{FF}}, \tag{20}$$

$$\tilde{\mathbf{x}}_i = \mathbf{x}_i - \eta \, \boldsymbol{\varepsilon}_{i-1} + \eta \, \boldsymbol{\varepsilon}_i \, B_i. \tag{21}$$

After L inference loops (with $\mathbf{x}_{i+1}$ and $\mathbf{x}_{i-1}$ held fixed), the FF weight is updated using the final $\tilde{\mathbf{x}}_i$ and $\boldsymbol{\varepsilon}_i$:

$$\Delta W_i^{\mathrm{FF}} = \alpha \, \tilde{\mathbf{x}}_i^{\top} \boldsymbol{\varepsilon}_i. \tag{22}$$

Substitution and expansion. Substituting (21) into (22):

$$\begin{aligned} \Delta W_i^{\mathrm{FF}} &= \alpha \big(\mathbf{x}_i - \eta \, \boldsymbol{\varepsilon}_{i-1} + \eta \, \boldsymbol{\varepsilon}_i \, B_i \big)^{\top} \boldsymbol{\varepsilon}_i \\ &= \alpha \, \mathbf{x}_i^{\top} \boldsymbol{\varepsilon}_i \;-\; \alpha\eta \, \boldsymbol{\varepsilon}_{i-1}^{\top} \boldsymbol{\varepsilon}_i \;+\; \alpha\eta \, (\boldsymbol{\varepsilon}_i \, B_i)^{\top} \boldsymbol{\varepsilon}_i. \end{aligned} \tag{23}$$

Expected drift under the whiteness approximation. The alignment drift contains the error auto-correlation $\boldsymbol{\varepsilon}_i^\top \boldsymbol{\varepsilon}_i$. For a single sample this is a rank-1 outer product with prominent off-diagonal structure. Averaged over a mini-batch, however, the batch-mean auto-correlation is diagonally dominant throughout training (Fig. 2D). We therefore make the approximation

$$\mathbb{E}\big[\boldsymbol{\varepsilon}_i^\top \boldsymbol{\varepsilon}_i\big] \approx \sigma_{\boldsymbol{\varepsilon}}^2 \, I, \tag{24}$$

where $\sigma_{\boldsymbol{\varepsilon}}^2$ is the mean squared error magnitude. This *whiteness approximation* is a batch-statistical property that holds from the first training step, not a consequence of learning (Sec. 5.2; Fig. 2D).

Taking the expectation of (16) and applying (24):

$$\mathbb{E}[\Delta W_i^{\mathrm{FF}}] = \alpha \, \mathbb{E}[\mathbf{x}_i^\top \boldsymbol{\varepsilon}_i] \; - \; \alpha\eta \, \mathbb{E}[\boldsymbol{\varepsilon}_{i-1}^\top \boldsymbol{\varepsilon}_i] \; + \; \alpha\eta \, \sigma_{\boldsymbol{\varepsilon}}^2 \, B_i^\top. \tag{25}$$

The third term is a persistent drift in the direction of $B_i^\top$: at every training step, regardless of the current value of W_i^{FF}, the inference loop nudges W_i^{FF} toward the transpose of the fixed random matrix B_i. This is the mechanism of Loop Alignment in PC-RFB.

PC-DH as the $B_i = W_i^{\mathrm{FB}}$ case. In PC-DH the FB matrix is not fixed; it plays the role of B_i at each step. The drift term becomes $\alpha\eta \, \sigma_{\boldsymbol{\varepsilon}}^2 \, (W_i^{\mathrm{FB}})^\top$, nudging W_i^{FF} toward $(W_i^{\mathrm{FB}})^\top$. Simultaneously, the symmetric Hebbian rule (Eq. 6) nudges W_i^{FB} toward $(W_i^{\mathrm{FF}})^\top$. Loop Alignment is therefore driven from *both* directions in PC-DH, which is consistent with the stronger alignment (cosine $\to$ 1.0) relative to PC-RFB (cosine $\to$ 0.9).

E Circuit sharing: relay structure and update rules

Relay matrices. The shared feedback pathway is parameterized by two relay matrices, $\mathbf{U} \in \mathbb{R}^{m \times n}$ and $\mathbf{V} \in \mathbb{R}^{m \times n}$, where n and m are the representation and error neuron counts of the shared layer. Both matrices are initialized Xavier-normal and are common to the feedback paths of both networks. The two effective FB weights traverse the closed loop in opposite directions (Eq. 17):

$$W_i^{\mathrm{FB,\,Net\,1}} = \mathbf{U} \, W_i^{\mathrm{Net\,2}} \, \mathbf{V}, \qquad W_i^{\mathrm{FB,\,Net\,2}} = \mathbf{V} \, W_i^{\mathrm{Net\,1}} \, \mathbf{U}. \tag{26}$$

Alternating training with one network active per step. Training proceeds in strict alternation, with no mini-batching. At every step exactly one of $\{\text{Net 1}, \text{Net 2}\}$ is the *active* network; the other serves only as a passive relay along the shared pathway. We describe the active-Net 1 step; the active-Net 2 step is symmetric (interchange labels, swap $\mathbf{U} \leftrightarrow \mathbf{V}$ in the loop-traversal direction).

Net 1-active step: inference. A single image is sampled and clamped as Net 1's bottom and top representations. All weights are held fixed during inference. Net 1's activities are updated for $L_{\mathrm{inf}} = 50$ loops by

$$\Delta \mathbf{x}_i^{\mathrm{Net\,1}} = \eta\big(-\boldsymbol{\varepsilon}_{i-1}^{\mathrm{Net\,1}} + \boldsymbol{\varepsilon}_i^{\mathrm{Net\,1}} \, \mathbf{U} \, W_i^{\mathrm{Net\,2}} \, \mathbf{V}\big). \tag{27}$$

Net 2 is not driven by any external input during this step, so its shared-layer variables are determined by Net 1's signal traversing the loop:

$$\mathbf{x}_i^{\mathrm{Net\,2}} = \boldsymbol{\varepsilon}_i^{\mathrm{Net\,1}} \, \mathbf{U}, \qquad \boldsymbol{\varepsilon}_i^{\mathrm{Net\,2}} = \boldsymbol{\varepsilon}_i^{\mathrm{Net\,1}} \, \mathbf{U} \, W_i^{\mathrm{Net\,2}}. \tag{28}$$

These are not independent state variables of Net 2; they are the post-$\mathbf{U}$ and post-$W^{\mathrm{Net\,2}}$ values of Net 1's signal, recomputed at every inference loop. After 50 loops, we read out the converged Net 1 activities together with the corresponding final relay values from Eq. (28).

Net 1-active step: weight updates. Only the weights belonging to the active network's path are updated. In the partial-plasticity setting (Figs. 3B–D), $\mathbf{U}$ and $\mathbf{V}$ are fixed and only $W_i^{\mathrm{Net\,1}}$ is updated:

$$\Delta W_i^{\mathrm{Net\,1}} = \alpha \, (\mathbf{x}_i^{\mathrm{Net\,1}})^\top \boldsymbol{\varepsilon}_i^{\mathrm{Net\,1}}. \tag{29}$$

In the fully plastic setting (Figs. 3E–G), $\mathbf{U}$ and $\mathbf{V}$ additionally receive local Hebbian updates at their respective synapses, using the active network's variables and the relay values from Eq. (28):

$$\Delta \mathbf{U}^{(\text{Net 1-active})} = \alpha \, (\boldsymbol{\varepsilon}_i^{\mathrm{Net\,1}})^\top \, \mathbf{x}_i^{\mathrm{Net\,2}} = \alpha \, (\boldsymbol{\varepsilon}_i^{\mathrm{Net\,1}})^\top (\boldsymbol{\varepsilon}_i^{\mathrm{Net\,1}} \, \mathbf{U}), \tag{30}$$

$$\Delta \mathbf{V}^{(\text{Net 1-active})} = \alpha \, (\boldsymbol{\varepsilon}_i^{\mathrm{Net\,2}})^\top \, \mathbf{x}_i^{\mathrm{Net\,1}} = \alpha \, (\boldsymbol{\varepsilon}_i^{\mathrm{Net\,1}} \, \mathbf{U} \, W_i^{\mathrm{Net\,2}})^\top \mathbf{x}_i^{\mathrm{Net\,1}}. \tag{31}$$

Equation (30) is the local Hebbian product at $\mathbf{U}$'s input/output synapses: pre-synaptic activity $\varepsilon_i^{\text{Net 1}}$ entering $\mathbf{U}$, post-synaptic activity $\mathbf{x}_i^{\text{Net 2}} = \varepsilon_i^{\text{Net 1}}\mathbf{U}$ exiting it. Equation (31) is the analogous product at $\mathbf{V}$: pre-synaptic activity $\varepsilon_i^{\text{Net 2}} = \varepsilon_i^{\text{Net 1}}\mathbf{U}\,W_i^{\text{Net 2}}$ entering $\mathbf{V}$, post-synaptic representation $\mathbf{x}_i^{\text{Net 1}}$ where the loop closes. The partner network's FF weight $W_i^{\text{Net 2}}$ is *not* updated on this step.

Net 2-active step. The next step interchanges roles. A single image is clamped to Net 2; Net 2's activities update via $\Delta\mathbf{x}_i^{\text{Net 2}} = \eta(-\varepsilon_{i-1}^{\text{Net 2}} + \varepsilon_i^{\text{Net 2}}\,\mathbf{V}\,W_i^{\text{Net 1}}\,\mathbf{U})$, and Net 1's shared-layer variables become the relay values $\mathbf{x}_i^{\text{Net 1}} = \varepsilon_i^{\text{Net 2}}\,\mathbf{V}$, $\varepsilon_i^{\text{Net 1}} = \varepsilon_i^{\text{Net 2}}\,\mathbf{V}\,W_i^{\text{Net 1}}$. The weight updates are the symmetric counterparts of Eqs. (29)–(31), with Net 1 and Net 2 indices swapped and the $\mathbf{U} \leftrightarrow \mathbf{V}$ traversal order reversed accordingly. The two data pools (MNIST and Fashion-MNIST) are shuffled with independent random seeds. The learning rate for all updates is α (Table 1).

F Algorithm descriptions

All models share the same inference loop structure; they differ only in the FB pathway and which weights are updated. Within each mini-batch of size 20, samples are processed *sequentially*: inference and weight update for sample k complete before sample $k+1$ begins. Algorithms 1–3 describe single-task learning; Algorithms 4–5 describe one alternating step of each circuit-sharing condition. For all algorithms, layer indices run over the non-clamped representation layers ($i = 1, \ldots, L-1$ for an L-layer network).

Algorithm 1 Baseline PC (one sample)

Require: input $\mathbf{x}_0$, label $\mathbf{x}_L$, weights W^{FF}, learning rates η, α

Clamp $\mathbf{x}_0 \leftarrow$ input, $\mathbf{x}_L \leftarrow$ label
Initialize $\mathbf{x}_i \leftarrow \mathbf{0}$ for $i = 1, \ldots, L-1$
for $\ell = 1$ to L_{inf} **do** ▷ inference loops ($L_{\text{inf}} = 50$)
  for $i = 0$ to $L-1$ **do**
    $\varepsilon_i \leftarrow \mathbf{x}_{i+1} - \mathbf{x}_i\,W_i^{\text{FF}}$
  end for
  for $i = 1$ to $L-1$ **do**
    $\mathbf{x}_i \leftarrow \mathbf{x}_i + \eta\big(-\varepsilon_{i-1} + \varepsilon_i\,(W_i^{\text{FF}})^\top\big)$
  end for
end for
for $i = 0$ to $L-1$ **do** ▷ weight update
  $\varepsilon_i \leftarrow \mathbf{x}_{i+1} - \mathbf{x}_i\,W_i^{\text{FF}}$
  $W_i^{\text{FF}} \leftarrow W_i^{\text{FF}} + \alpha\,\mathbf{x}_i^\top\varepsilon_i$
end for

Algorithm 2 PC-DH (one sample)

Require: input $\mathbf{x}_0$, label $\mathbf{x}_L$, weights W^{FF}, W^{FB}, learning rates η, α

Clamp $\mathbf{x}_0, \mathbf{x}_L$; initialize $\mathbf{x}_i \leftarrow \mathbf{0}$
for $\ell = 1$ to L_{inf} **do**
  for $i = 0$ to $L-1$ **do**
    $\varepsilon_i \leftarrow \mathbf{x}_{i+1} - \mathbf{x}_i\,W_i^{\text{FF}}$
  end for
  for $i = 1$ to $L-1$ **do**
    $\mathbf{x}_i \leftarrow \mathbf{x}_i + \eta(-\varepsilon_{i-1} + \varepsilon_i\,W_i^{\text{FB}})$
  end for
end for
$\varepsilon_i \leftarrow \mathbf{x}_{i+1} - \mathbf{x}_i\,W_i^{\text{FF}}$ for all i
for $i = 0$ to $L-1$ **do** ▷ independent Hebbian updates
  $W_i^{\text{FF}} \leftarrow W_i^{\text{FF}} + \alpha\,\mathbf{x}_i^\top\varepsilon_i$
  $W_i^{\text{FB}} \leftarrow W_i^{\text{FB}} + \alpha\,\varepsilon_i^\top\mathbf{x}_i$
end for

Algorithm 3 PC-RFB (one sample)

Require: input $\mathbf{x}_0$, label $\mathbf{x}_L$, FF weight W^{FF}, fixed random B (never updated)

Clamp $\mathbf{x}_0$, $\mathbf{x}_L$; initialize $\mathbf{x}_i \leftarrow \mathbf{0}$
for $\ell = 1$ to L_{inf} **do**
  for $i = 0$ to $L-1$ **do**
    $\boldsymbol{\varepsilon}_i \leftarrow \mathbf{x}_{i+1} - \mathbf{x}_i W_i^{\mathrm{FF}}$
  end for
  for $i = 1$ to $L-1$ **do**
    $\mathbf{x}_i \leftarrow \mathbf{x}_i + \eta(-\boldsymbol{\varepsilon}_{i-1} + \boldsymbol{\varepsilon}_i B_i)$
  end for
end for
$\boldsymbol{\varepsilon}_i \leftarrow \mathbf{x}_{i+1} - \mathbf{x}_i W_i^{\mathrm{FF}}$ for all i
$W_i^{\mathrm{FF}} \leftarrow W_i^{\mathrm{FF}} + \alpha\, \mathbf{x}_i^{\top} \boldsymbol{\varepsilon}_i$ ▷ B_i unchanged

Algorithm 4 Circuit sharing, partial plasticity — one alternating pair of steps

Require: FF weights $W^{\mathrm{Net}\,1}$, $W^{\mathrm{Net}\,2}$; fixed relays $\mathbf{U}$, $\mathbf{V}$; data pools $\mathcal{D}^1$ (MNIST), $\mathcal{D}^2$ (Fashion-MNIST)

// Net 1-active step (Net 2 is passive relay)
Sample $s^{\mathrm{Net}\,1} \sim \mathcal{D}^1$
Clamp $\mathbf{x}_0^{\mathrm{Net}\,1}, \mathbf{x}_L^{\mathrm{Net}\,1}$ from $s^{\mathrm{Net}\,1}$; initialize $\mathbf{x}_i^{\mathrm{Net}\,1} \leftarrow \mathbf{0}$
for $\ell = 1$ to L_{inf} **do** ▷ Net 1 inference, Net 2 as relay
  for $i = 0$ to $L-1$ **do**
    $\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1} \leftarrow \mathbf{x}_{i+1}^{\mathrm{Net}\,1} - \mathbf{x}_i^{\mathrm{Net}\,1} W_i^{\mathrm{Net}\,1}$
  end for
  $\mathbf{x}_i^{\mathrm{Net}\,2} \leftarrow \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1}\mathbf{U}$, $\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2} \leftarrow \mathbf{x}_i^{\mathrm{Net}\,2} W_i^{\mathrm{Net}\,2}$ ▷ relay values, Eq. (28)
  for $i = 1$ to $L-1$ **do**
    $\mathbf{x}_i^{\mathrm{Net}\,1} \leftarrow \mathbf{x}_i^{\mathrm{Net}\,1} + \eta\big(-\boldsymbol{\varepsilon}_{i-1}^{\mathrm{Net}\,1} + \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1}\mathbf{U}\, W_i^{\mathrm{Net}\,2}\mathbf{V}\big)$
  end for
end for
$\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1} \leftarrow \mathbf{x}_{i+1}^{\mathrm{Net}\,1} - \mathbf{x}_i^{\mathrm{Net}\,1} W_i^{\mathrm{Net}\,1}$ for all i
$W_i^{\mathrm{Net}\,1} \leftarrow W_i^{\mathrm{Net}\,1} + \alpha\,(\mathbf{x}_i^{\mathrm{Net}\,1})^{\top}\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1}$ ▷ $\mathbf{U}, \mathbf{V}, W^{\mathrm{Net}\,2}$ unchanged

// Net 2-active step (Net 1 is passive relay)
Sample $s^{\mathrm{Net}\,2} \sim \mathcal{D}^2$
Clamp $\mathbf{x}_0^{\mathrm{Net}\,2}, \mathbf{x}_L^{\mathrm{Net}\,2}$ from $s^{\mathrm{Net}\,2}$; initialize $\mathbf{x}_i^{\mathrm{Net}\,2} \leftarrow \mathbf{0}$
for $\ell = 1$ to L_{inf} **do** ▷ Net 2 inference, Net 1 as relay
  for $i = 0$ to $L-1$ **do**
    $\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2} \leftarrow \mathbf{x}_{i+1}^{\mathrm{Net}\,2} - \mathbf{x}_i^{\mathrm{Net}\,2} W_i^{\mathrm{Net}\,2}$
  end for
  $\mathbf{x}_i^{\mathrm{Net}\,1} \leftarrow \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2}\mathbf{V}$, $\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1} \leftarrow \mathbf{x}_i^{\mathrm{Net}\,1} W_i^{\mathrm{Net}\,1}$ ▷ relay values
  for $i = 1$ to $L-1$ **do**
    $\mathbf{x}_i^{\mathrm{Net}\,2} \leftarrow \mathbf{x}_i^{\mathrm{Net}\,2} + \eta\big(-\boldsymbol{\varepsilon}_{i-1}^{\mathrm{Net}\,2} + \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2}\mathbf{V}\, W_i^{\mathrm{Net}\,1}\mathbf{U}\big)$
  end for
end for
$\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2} \leftarrow \mathbf{x}_{i+1}^{\mathrm{Net}\,2} - \mathbf{x}_i^{\mathrm{Net}\,2} W_i^{\mathrm{Net}\,2}$ for all i
$W_i^{\mathrm{Net}\,2} \leftarrow W_i^{\mathrm{Net}\,2} + \alpha\,(\mathbf{x}_i^{\mathrm{Net}\,2})^{\top}\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2}$ ▷ $\mathbf{U}, \mathbf{V}, W^{\mathrm{Net}\,1}$ unchanged

Algorithm 5 Circuit sharing, full plasticity — one alternating pair of steps

Require: FF weights $W^{\mathrm{Net}\,1}$, $W^{\mathrm{Net}\,2}$; learnable relays $\mathbf{U}$, $\mathbf{V}$; data pools $\mathcal{D}^1$ (MNIST), $\mathcal{D}^2$ (Fashion-MNIST)

// Net 1-active step

Sample $s^{\mathrm{Net}\,1} \sim \mathcal{D}^1$; **Clamp** $\mathbf{x}_0^{\mathrm{Net}\,1}, \mathbf{x}_L^{\mathrm{Net}\,1}$; initialize $\mathbf{x}_i^{\mathrm{Net}\,1} \leftarrow \mathbf{0}$
for $\ell = 1$ to L_{inf} **do** ▷ Net 1 inference, Net 2 as relay
  for $i = 0$ to $L-1$ **do**
    $\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1} \leftarrow \mathbf{x}_{i+1}^{\mathrm{Net}\,1} - \mathbf{x}_i^{\mathrm{Net}\,1} W_i^{\mathrm{Net}\,1}$
  end for
  $\mathbf{x}_i^{\mathrm{Net}\,2} \leftarrow \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1} \mathbf{U}$, $\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2} \leftarrow \mathbf{x}_i^{\mathrm{Net}\,2} W_i^{\mathrm{Net}\,2}$ ▷ relay values
  for $i = 1$ to $L-1$ **do**
    $\mathbf{x}_i^{\mathrm{Net}\,1} \leftarrow \mathbf{x}_i^{\mathrm{Net}\,1} + \eta\big(-\boldsymbol{\varepsilon}_{i-1}^{\mathrm{Net}\,1} + \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1} \mathbf{U} W_i^{\mathrm{Net}\,2} \mathbf{V}\big)$
  end for
end for
$\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1} \leftarrow \mathbf{x}_{i+1}^{\mathrm{Net}\,1} - \mathbf{x}_i^{\mathrm{Net}\,1} W_i^{\mathrm{Net}\,1}$ for all i; refresh $\mathbf{x}_i^{\mathrm{Net}\,2}, \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2}$ via Eq. (28)
$W_i^{\mathrm{Net}\,1} \leftarrow W_i^{\mathrm{Net}\,1} + \alpha\,(\mathbf{x}_i^{\mathrm{Net}\,1})^\top \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1}$
$\mathbf{U} \leftarrow \mathbf{U} + \alpha\,(\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1})^\top \mathbf{x}_i^{\mathrm{Net}\,2}$ ▷ Eq. (30)
$\mathbf{V} \leftarrow \mathbf{V} + \alpha\,(\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2})^\top \mathbf{x}_i^{\mathrm{Net}\,1}$ ▷ Eq. (31); $W^{\mathrm{Net}\,2}$ unchanged

// Net 2-active step (symmetric)

Sample $s^{\mathrm{Net}\,2} \sim \mathcal{D}^2$; **Clamp** $\mathbf{x}_0^{\mathrm{Net}\,2}, \mathbf{x}_L^{\mathrm{Net}\,2}$; initialize $\mathbf{x}_i^{\mathrm{Net}\,2} \leftarrow \mathbf{0}$
for $\ell = 1$ to L_{inf} **do** ▷ Net 2 inference, Net 1 as relay
  for $i = 0$ to $L-1$ **do**
    $\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2} \leftarrow \mathbf{x}_{i+1}^{\mathrm{Net}\,2} - \mathbf{x}_i^{\mathrm{Net}\,2} W_i^{\mathrm{Net}\,2}$
  end for
  $\mathbf{x}_i^{\mathrm{Net}\,1} \leftarrow \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2} \mathbf{V}$, $\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1} \leftarrow \mathbf{x}_i^{\mathrm{Net}\,1} W_i^{\mathrm{Net}\,1}$ ▷ relay values
  for $i = 1$ to $L-1$ **do**
    $\mathbf{x}_i^{\mathrm{Net}\,2} \leftarrow \mathbf{x}_i^{\mathrm{Net}\,2} + \eta\big(-\boldsymbol{\varepsilon}_{i-1}^{\mathrm{Net}\,2} + \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2} \mathbf{V} W_i^{\mathrm{Net}\,1} \mathbf{U}\big)$
  end for
end for
$\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2} \leftarrow \mathbf{x}_{i+1}^{\mathrm{Net}\,2} - \mathbf{x}_i^{\mathrm{Net}\,2} W_i^{\mathrm{Net}\,2}$ for all i; refresh $\mathbf{x}_i^{\mathrm{Net}\,1}, \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1}$
$W_i^{\mathrm{Net}\,2} \leftarrow W_i^{\mathrm{Net}\,2} + \alpha\,(\mathbf{x}_i^{\mathrm{Net}\,2})^\top \boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2}$
$\mathbf{U} \leftarrow \mathbf{U} + \alpha\,(\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,2})^\top \mathbf{x}_i^{\mathrm{Net}\,1}$ ▷ Net 2-active $\Delta\mathbf{U}$
$\mathbf{V} \leftarrow \mathbf{V} + \alpha\,(\boldsymbol{\varepsilon}_i^{\mathrm{Net}\,1})^\top \mathbf{x}_i^{\mathrm{Net}\,2}$ ▷ Net 2-active $\Delta\mathbf{V}$; $W^{\mathrm{Net}\,1}$ unchanged

G Computational resources

All experiments were run on a single CPU (no GPU acceleration). Main single-task experiments (10^6 steps, 50 independent sessions per model) required approximately 10 minutes per session, or roughly 25 CPU-hours per model variant. Circuit-sharing and robustness experiments (3×10^6 steps, 10 sessions) required approximately 30 minutes per session. The full robustness sweep (7 noise levels, 10 sessions each) required approximately 35 CPU-hours in total. No hyperparameter search was performed; all values listed in Table 1 were fixed a priori.